BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI **Semester:** 5 **Academic Year:** 2026-27
Division: C **Batch:** 3 **Practical No:** 10
Roll no: 59 **PRN No:** 12413745 **Name of Student:** Netra Lalwani
Subject Name: Deep Learning

Problem Statement:

Design and implement a deep learning application for audio classification that incorporates Speech-to-Text (STT) and Text-to-Speech (TTS) using an appropriate deep learning architecture, and evaluate its performance using suitable metrics.

Algorithm:

Spoken-Language Identification, Speech-to-Text and Text-to-Speech on Indian Languages (CRNN + Whisper-small + MMS-TTS) using the FLEURS Dataset

1. Start the program.
2. Install and import the required libraries (PyTorch, torchaudio, Transformers, jiwer, uroman, librosa, scikit-learn, Matplotlib, Seaborn, etc.) and select the GPU device.
3. Download the Google FLEURS validation split (Parquet files) for Hindi, Marathi, Tamil and Telugu from the Hugging Face Hub.
4. Keep 200 utterances per language; decode the audio, resample to 16 kHz, trim silence and peak-normalize the volume.
5. Split the data into train, validation and test sets with StratifiedGroupKFold, grouping by sentence id so the same sentence never appears in two sets.
6. Explore the data: utterances per language, duration distribution, waveforms and log-mel spectrograms.
7. Build the CRNN classifier: 64-band log-mel spectrogram, 3 convolution blocks, a bidirectional GRU and a linear layer, with SpecAugment during training.
8. Train the CRNN for 25 epochs (AdamW, cosine learning-rate schedule, label smoothing) and keep the weights with the best validation macro-F1.
9. Evaluate the classifier on the test set using accuracy, precision, recall, F1-score, a confusion matrix, accuracy versus clip length and a t-SNE plot of the embeddings.
10. Pass the CRNN-predicted language to Whisper-small to transcribe the speech (Speech-to-Text) and compute Word Error Rate (WER) and Character Error Rate (CER).
11. Synthesize speech from text using MMS-TTS (VITS) (Text-to-Speech) and evaluate it with round-trip CER (Whisper run on the synthetic audio) and duration.
12. Run the complete pipeline on a test clip: identify the language, transcribe it and speak it back.
13. Display the results summary.
14. End the program.

Code:

```
!pip install -q jiwer uroman

import io, time, random, unicodedata, warnings
import numpy as np, pandas as pd
import matplotlib.pyplot as plt, seaborn as sns
import soundfile as sf, librosa, librosa.display
import torch, torch.nn as nn
import torchaudio
from torch.utils.data import Dataset, DataLoader
from huggingface_hub import hf_hub_download
from sklearn.model_selection import StratifiedGroupKFold
from sklearn.metrics import accuracy_score, f1_score, classification_report, confusion_matrix
from sklearn.manifold import TSNE
from IPython.display import Audio, HTML, display
import jiwer
from transformers import WhisperProcessor, WhisperForConditionalGeneration, VitsModel, AutoTokenizer
warnings.filterwarnings("ignore")

SEED = 42
random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device:", device)

SR, CROP = 16000, 4.0                     # sample rate, seconds per classifier input
# FLEURS code : (name, Whisper code, MMS-TTS code)
LANGS = {"hi_in": ("Hindi", "hi", "hin"), "mr_in": ("Marathi", "mr", "mar"),
         "ta_in": ("Tamil", "ta", "tam"), "te_in": ("Telugu", "te", "tel")}
CODES = list(LANGS); NAMES = [LANGS[c][0] for c in CODES]; N_CLS = len(CODES)
COLORS = dict(zip(NAMES, sns.color_palette("Set2", N_CLS)))
sns.set_theme(style="whitegrid")

def crop(w, sec=CROP, train=False):
    n = int(sec * SR)
    if len(w) >= n:
        s = random.randint(0, len(w) - n) if train else (len(w) - n) // 2
        return w[s:s + n].astype(np.float32)
    return np.pad(w, (0, n - len(w))).astype(np.float32)

def norm_text(t):            # for WER/CER: remove punctuation, keep Indic vowel signs
    t = unicodedata.normalize("NFC", str(t)).lower()
    return " ".join("".join(" " if unicodedata.category(c)[0] in "PS" else c for c in t).split())

WAVS, rows = [], []
for code in CODES:
    path = hf_hub_download("google/fleurs", f"parquet-data/{code}/validation-00000-of-00001.parquet",
        repo_type="dataset")
    df = pd.read_parquet(path, columns=["id", "audio", "transcription", "raw_transcription"]).sample(frac=1,
        random_state=SEED).head(200)
    for r in df.itertuples(index=False):
        w, sr = sf.read(io.BytesIO(r.audio["bytes"]), dtype="float32")
        if w.ndim > 1: w = w.mean(1)
        if sr != SR: w = librosa.resample(w, orig_sr=sr, target_sr=SR)
        w, _ = librosa.effects.trim(w, top_db=35)
        WAVS.append((w / (np.abs(w).max() + 1e-8) * 0.9).astype(np.float32))
        rows.append(dict(lang=code, language=LANGS[code][0], sent_id=int(r.id), text=r.raw_transcription,
                         transcription=r.transcription, dur=len(w) / SR))
    print(f"{LANGS[code][0]:8s}: {len(df)} utterances")
meta = pd.DataFrame(rows); meta["y"] = meta.lang.map(CODES.index); Y = meta.y.values

# split by sentence id, so the same sentence never appears in both train and test
sgkf = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=SEED)
trval, test_idx = next(sgkf.split(meta, Y, groups=meta.sent_id))
a, b = next(StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=SEED).split(meta.iloc[trval], Y[trval],
    groups=meta.sent_id.values[trval]))
train_idx, val_idx = trval[a], trval[b]
print(f"\nTrain {len(train_idx)} | Val {len(val_idx)} | Test {len(test_idx)}")
meta.head()

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
meta.language.value_counts().reindex(NAMES).plot.bar(ax=axes[0], color=[COLORS[n] for n in NAMES], rot=0)
axes[0].set_title("Utterances per language")
sns.boxplot(data=meta, x="language", y="dur", hue="language", order=NAMES, palette=COLORS, legend=False, ax=axes[1])
axes[1].set_title("Utterance duration (seconds)"); axes[1].set_xlabel("")
plt.tight_layout(); plt.show()

fig, axes = plt.subplots(2, N_CLS, figsize=(18, 6))
for k, code in enumerate(CODES):
    w = WAVS[meta.index[meta.lang == code][0]][:4 * SR]
    librosa.display.waveshow(w, sr=SR, ax=axes[0, k], color=COLORS[NAMES[k]]); axes[0, k].set_title(f"{NAMES[k]} —
        waveform")
    S = librosa.power_to_db(librosa.feature.melspectrogram(y=w, sr=SR, n_mels=64, n_fft=400, hop_length=160),
        ref=np.max)
```

```
    librosa.display.specshow(S, sr=SR, hop_length=160, x_axis="time", y_axis="mel", ax=axes[1, k], cmap="magma")
    axes[1, k].set_title(f"{NAMES[k]} — log-mel spectrogram")
plt.tight_layout(); plt.show()

# Listen to one sample per language
for code in CODES:
    i = meta.index[meta.lang == code][0]
    display(HTML(f"<b>{LANGS[code][0]}</b>: {meta.text[i]}")); display(Audio(WAVS[i], rate=SR))

class CRNN(nn.Module):
    def __init__(self, n_cls, n_mels=64, hid=128):
        super().__init__()
        self.mel = torchaudio.transforms.MelSpectrogram(SR, n_fft=400, hop_length=160, n_mels=n_mels)
        self.aug = nn.Sequential(torchaudio.transforms.FrequencyMasking(10), torchaudio.transforms.TimeMasking(40))
        def block(ci, co): return nn.Sequential(nn.Conv2d(ci, co, 3, padding=1), nn.BatchNorm2d(co), nn.ReLU(),
            nn.MaxPool2d(2))
        self.cnn = nn.Sequential(block(1, 32), block(32, 64), block(64, 128))          # 64 mels -> 8
        self.gru = nn.GRU(128 * (n_mels // 8), hid, batch_first=True, bidirectional=True)
        self.fc = nn.Sequential(nn.Dropout(0.3), nn.Linear(2 * hid, n_cls))
    def spec(self, x):
        m = torch.log(self.mel(x) + 1e-6)
        m = (m - m.mean(dim=(1, 2), keepdim=True)) / (m.std(dim=(1, 2), keepdim=True) + 1e-5)
        return self.aug(m) if self.training else m
    def embed(self, x):
        h = self.cnn(self.spec(x).unsqueeze(1))                                        # (B, C, F, T)
        B, C, Fq, T = h.shape
        out, _ = self.gru(h.permute(0, 3, 1, 2).reshape(B, T, C * Fq))
        return out.mean(1)
    def forward(self, x): return self.fc(self.embed(x))

class WaveDS(Dataset):
    def __init__(self, idx, train): self.idx, self.train = idx, train
    def __len__(self): return len(self.idx)
    def __getitem__(self, i):
        j = self.idx[i]; return torch.from_numpy(crop(WAVS[j], CROP, self.train)), int(Y[j])

train_dl = DataLoader(WaveDS(train_idx, True), batch_size=32, shuffle=True, num_workers=2)
model = CRNN(N_CLS).to(device)
print(f"CRNN parameters: {sum(p.numel() for p in model.parameters())/1e6:.2f} M")

@torch.no_grad()
def predict(wavs, sec=CROP):
    model.eval(); x = torch.stack([torch.from_numpy(crop(w, sec)) for w in wavs]).to(device)
    return torch.cat([model(x[i:i + 64]).softmax(-1) for i in range(0, len(x), 64)]).cpu().numpy()

EPOCHS = 25
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-2)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, EPOCHS)
crit = nn.CrossEntropyLoss(label_smoothing=0.1)
hist, best_f1 = [], 0
for ep in range(1, EPOCHS + 1):
    model.train(); tot, correct, n = 0, 0, 0
    for x, y in train_dl:
        x, y = x.to(device), y.to(device)
        out = model(x); loss = crit(out, y)
        opt.zero_grad(); loss.backward(); opt.step()
        tot += loss.item() * len(y); correct += (out.argmax(1) == y).sum().item(); n += len(y)
    sched.step()
    pv = predict([WAVS[j] for j in val_idx]).argmax(1)
    va, vf = accuracy_score(Y[val_idx], pv), f1_score(Y[val_idx], pv, average="macro")
    hist.append(dict(epoch=ep, loss=tot / n, train_acc=correct / n, val_acc=va, val_f1=vf))
    if vf > best_f1: best_f1, best_state = vf, {k: v.clone() for k, v in model.state_dict().items()}
    print(f"epoch {ep:2d} | loss {tot/n:.3f} | train acc {correct/n:.3f} | val acc {va:.3f} | val F1 {vf:.3f}")
model.load_state_dict(best_state)
hist = pd.DataFrame(hist)

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
axes[0].plot(hist.epoch, hist.loss, "o-"); axes[0].set_title("Training loss"); axes[0].set_xlabel("epoch")
axes[1].plot(hist.epoch, hist.train_acc, "o-", label="train acc"); axes[1].plot(hist.epoch, hist.val_acc, "s-",
    label="val acc")
axes[1].plot(hist.epoch, hist.val_f1, "^-", label="val macro-F1"); axes[1].legend(); axes[1].set_title("Accuracy");
    axes[1].set_xlabel("epoch")
plt.tight_layout(); plt.show()

test_w, y_test = [WAVS[j] for j in test_idx], Y[test_idx]
P = predict(test_w); y_pred = P.argmax(1)
print(f"Test accuracy: {accuracy_score(y_test, y_pred):.4f} | Macro-F1: {f1_score(y_test, y_pred,
    average='macro'):.4f}\n")
print(classification_report(y_test, y_pred, target_names=NAMES, digits=4))

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
sns.heatmap(confusion_matrix(y_test, y_pred, normalize="true"), annot=True, fmt=".2f", cmap="Blues",
            xticklabels=NAMES, yticklabels=NAMES, ax=axes[0])
axes[0].set_title("Normalised confusion matrix"); axes[0].set_xlabel("Predicted"); axes[0].set_ylabel("True")
```

```
# Extra: how much speech does the model need?
secs = [1, 2, 3, 4, 6]
accs = [accuracy_score(y_test, predict(test_w, s).argmax(1)) for s in secs]
axes[1].plot(secs, accs, "o-", lw=2.5, color="#e67e22")
for s, a in zip(secs, accs): axes[1].text(s, a + .01, f"{a:.2f}", ha="center")
axes[1].set_ylim(0, 1.05); axes[1].set_xlabel("seconds of audio"); axes[1].set_ylabel("test accuracy")
axes[1].set_title("Accuracy vs. length of the audio clip")
plt.tight_layout(); plt.show()

# Extra: t-SNE of the learned CRNN embeddings (Indo-Aryan: Hindi, Marathi | Dravidian: Tamil, Telugu)
with torch.no_grad():
    model.eval()
    E = torch.cat([model.embed(torch.stack([torch.from_numpy(crop(w)) for w in test_w[i:i + 64]]).to(device)).cpu()
                   for i in range(0, len(test_w), 64)]).numpy()
Z = TSNE(n_components=2, perplexity=25, random_state=SEED, init="pca").fit_transform(E)
plt.figure(figsize=(7, 5.5))
for k, n in enumerate(NAMES):
    plt.scatter(Z[y_test == k, 0], Z[y_test == k, 1], s=18, color=COLORS[n], label=n)
plt.legend(); plt.xticks([]); plt.yticks([]); plt.title("t-SNE of CRNN embeddings (test set)"); plt.show()

asr_proc = WhisperProcessor.from_pretrained("openai/whisper-small")
asr = WhisperForConditionalGeneration.from_pretrained("openai/whisper-small").to(device).eval()

@torch.no_grad()
def transcribe(wav, lang_code):
    feats = asr_proc(wav[:30 * SR], sampling_rate=SR, return_tensors="pt").input_features.to(device)
    ids = asr.generate(input_features=feats, language=lang_code, task="transcribe", max_new_tokens=400)
    return asr_proc.batch_decode(ids, skip_special_tokens=True)[0].strip()

def err(fn, ref, hyp):
    ref, hyp = norm_text(ref), norm_text(hyp)
    return float(fn(ref, hyp)) if ref else np.nan

stt_rows = []
for k in range(N_CLS):
    for pos in np.where(y_test == k)[0][:10]:                          # 10 test clips per language
        j = test_idx[pos]; pred_code = CODES[y_pred[pos]]
        hyp = transcribe(WAVS[j], LANGS[pred_code][1])
        stt_rows.append(dict(language=NAMES[k], predicted=LANGS[pred_code][0], reference=meta.transcription[j],
            whisper=hyp,
                             WER=err(jiwer.wer, meta.transcription[j], hyp), CER=err(jiwer.cer, meta.transcription[j],
                                 hyp)))
stt = pd.DataFrame(stt_rows)
summary = stt.groupby("language")[["WER", "CER"]].mean().reindex(NAMES)
summary.loc["Average"] = summary.mean()
display(summary.round(3))

summary.drop("Average").plot.bar(figsize=(9, 4), rot=0, color=["#3498db", "#e74c3c"])
plt.title("Whisper-small: WER and CER per language (lower is better)"); plt.ylabel("error rate"); plt.show()
display(stt[["language", "reference", "whisper", "CER"]].groupby("language").head(2))

try:
    import uroman as ur; UROMAN = ur.Uroman()
except Exception:
    UROMAN = None

@torch.no_grad()
def tts(text, code):
    mid = f"facebook/mms-tts-{LANGS[code][2]}"
    tok, vits = AutoTokenizer.from_pretrained(mid), VitsModel.from_pretrained(mid).to(device).eval()
    text = norm_text(text)
    if getattr(tok, "is_uroman", False) and UROMAN: text = UROMAN.romanize_string(text)
    torch.manual_seed(SEED)
    w = vits(**tok(text, return_tensors="pt").to(device)).waveform[0].cpu().numpy()
    if vits.config.sampling_rate != SR: w = librosa.resample(w, orig_sr=vits.config.sampling_rate, target_sr=SR)
    return w / (np.abs(w).max() + 1e-8) * 0.9

tts_rows, pairs = [], []
for k, code in enumerate(CODES):
    j = test_idx[np.where(y_test == k)[0][0]]
    syn = tts(meta.text[j], code)
    back = transcribe(syn, LANGS[code][1])
    tts_rows.append(dict(language=NAMES[k], real_sec=meta.dur[j], synthetic_sec=len(syn) / SR,
                         CER_real=err(jiwer.cer, meta.transcription[j], transcribe(WAVS[j], LANGS[code][1])),
                         CER_synthetic=err(jiwer.cer, meta.transcription[j], back)))
    pairs.append((NAMES[k], j, syn))
tts_df = pd.DataFrame(tts_rows); display(tts_df.round(3))

fig, axes = plt.subplots(2, N_CLS, figsize=(18, 6))
for c, (name, j, syn) in enumerate(pairs):
    for r, (w, lab) in enumerate([(WAVS[j], "human"), (syn, "MMS-TTS")]):
        S = librosa.power_to_db(librosa.feature.melspectrogram(y=w[:6 * SR], sr=SR, n_mels=64, n_fft=400,
            hop_length=160), ref=np.max)
        librosa.display.specshow(S, sr=SR, hop_length=160, x_axis="time", y_axis="mel", ax=axes[r, c], cmap="magma" if
```

```
            r == 0 else "viridis")
        axes[r, c].set_title(f"{name} — {lab}")
plt.suptitle("Same sentence: human recording (top) vs synthetic speech (bottom)", y=1.02); plt.tight_layout();
    plt.show()

tts_df.set_index("language")[["CER_real", "CER_synthetic"]].plot.bar(figsize=(9, 4), rot=0, color=["#34495e",
    "#1abc9c"])
plt.title("Round-trip intelligibility: Whisper CER on human vs synthetic speech"); plt.ylabel("CER"); plt.show()

# Listen: human vs synthetic
for name, j, syn in pairs:
    display(HTML(f"<hr><b>{name}</b>: {meta.text[j]}"))
    display(HTML("Human:")); display(Audio(WAVS[j], rate=SR))
    display(HTML("MMS-TTS:")); display(Audio(syn, rate=SR))

def pipeline(wav):
    p = predict([wav])[0]; code = CODES[p.argmax()]
    text = transcribe(wav, LANGS[code][1])
    return LANGS[code][0], p.max(), text, tts(text, code)

j = test_idx[np.where(y_test == 2)[0][-1]]          # a Tamil test clip
lang, conf, text, out = pipeline(WAVS[j])
display(HTML(f"<b>True language:</b> {meta.language[j]}   <b>Predicted:</b> {lang}
    ({conf:.1%})<br><b>Transcript:</b> {text}"))
display(HTML("Input:")); display(Audio(WAVS[j], rate=SR))
display(HTML("Spoken back:")); display(Audio(out, rate=SR))

print("Audio classification (CRNN, 4 s clips)")
print(f"  Test accuracy : {accuracy_score(y_test, y_pred):.4f}")
print(f"  Macro-F1      : {f1_score(y_test, y_pred, average='macro'):.4f}")
print("\nSpeech-to-text (Whisper-small, language from CRNN)")
print(f"  Average WER   : {summary.loc['Average', 'WER']:.3f}")
print(f"  Average CER   : {summary.loc['Average', 'CER']:.3f}")
print("\nText-to-speech (MMS-TTS)")
print(f"  Round-trip CER: human {tts_df.CER_real.mean():.3f} vs synthetic {tts_df.CER_synthetic.mean():.3f}")
```

Output :

1. Setup and dataset loading

```
Device: cuda

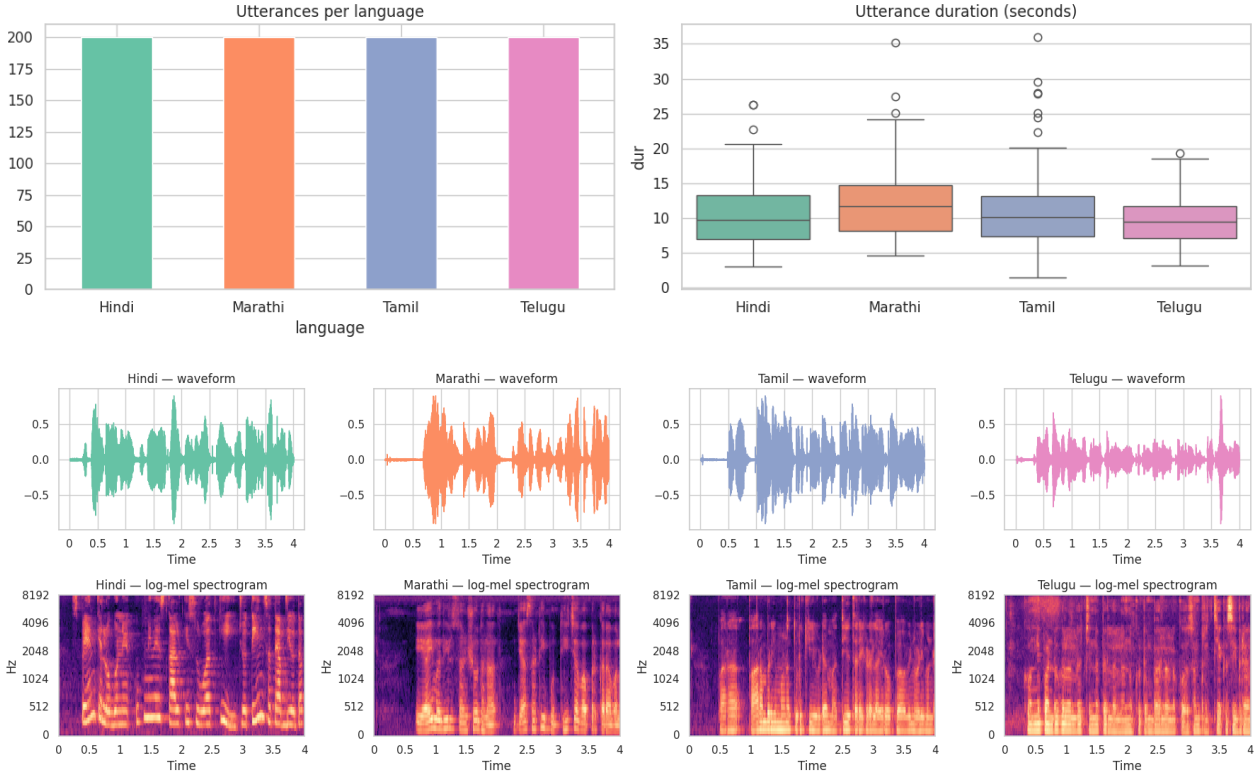
Hindi   : 200 utterances
Marathi : 200 utterances
Tamil   : 200 utterances
Telugu  : 200 utterances

Train 526 | Val 118 | Test 156
    lang language  sent_id                                                text  \
0  hi_in    Hindi     1636  □□□□□□ □□ □□□□□□ □□ □□□□□□□□ □□□□ □□ □□□□□□□ □□, □□...
1  hi_in    Hindi     1609  □□□□□□ □□□□□□□ □□ □□□□□ 41 □□□□□□□ □□□□ 11 □□□□□□ □□...
2  hi_in    Hindi     1604  □□□□□□ □□ □□□□ □□□□, □□□□□□□□□ □□□□□□□□□ □□□□ □□ □□□□...
3  hi_in    Hindi     1640  □□□□□□□□ □□ □□□□□□ □□□□□□ □□, □□□□□□ □□ □□□□□ □□ □□...
4  hi_in    Hindi     1567  □□□□ □□ □□□□□ □□□□□ □□□□□□□□□□□ □□□□□□□ □□□□ □□□□ □□□...

                                          transcription     dur  y
0  □□□□□□ □□ □□□□□□ □□ □□□□□□□□ □□□□ □□ □□□□□□□ □□ □□ ...   5.216  0
1  □□□□□□ □□□□□□□ □□ □□□□□ 41 □□□□□□□ □□□□ 11 □□□□□□ □□...   8.128  0
2  □□□□□□ □□ □□□□ □□□□ □□□□□□□□□ □□□□□□□□□ □□□□ □□ □□□□□...  26.280  0
3  □□□□□□□□ □□ □□□□□□ □□□□□□□ □□ □□□□□□ □□ □□□□□ □□ □□...   9.104  0
4  □□□□ □□ □□□□□ □□□□□ □□□□□□□□□□□ □□□□□□□ □□□□ □□□□ □□□...   4.736  0
```

	lang	language	sent_id	text	transcription	dur	y
0	hi_in	Hindi	1636	स्पेन के लोगों ने उपनिवेश ...	स्पेन के लोगों ने उपनिवेश ...	5.216	0
1	hi_in	Hindi	1609	रॉबिन उथप्पा ने केवल 41 गे...	रॉबिन उथप्पा ने केवल 41 गे...	8.128	0
2	hi_in	Hindi	1604	एनएसए के रूप में, उन्होंने...	एनएसए के रूप में उन्होंने ...	26.28	0
3	hi_in	Hindi	1640	बर्फीले और हिममय सड़कों पर,...	बर्फीले और हिममय सड़कों पर...	9.104	0
4	hi_in	Hindi	1567	साल का सबसे बड़ा टूर्नामें...	साल का सबसे बड़ा टूर्नामें...	4.736	0

2. Exploratory data analysis

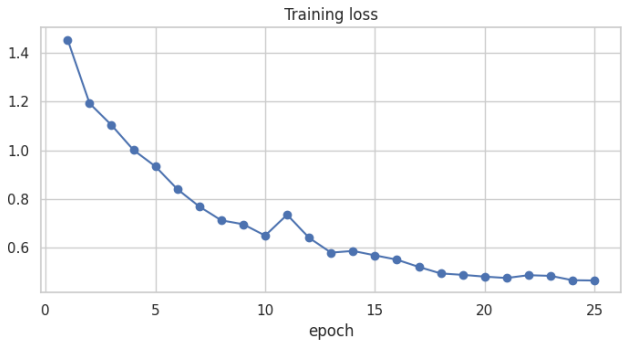


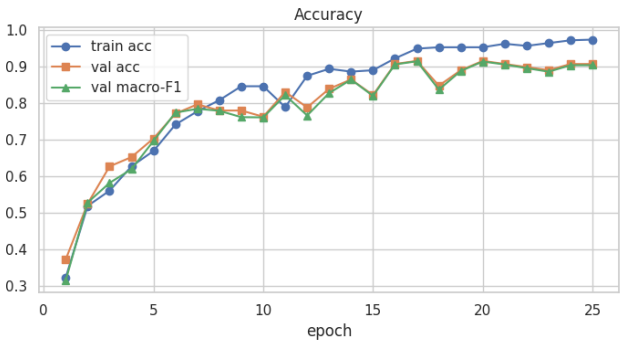
(Section 3 of the notebook also plays one audio clip per language with an audio player; not reproducible in a PDF.)

3. CRNN model and training

```
CRNN parameters: 0.98 M

epoch  1 | loss 1.454 | train acc 0.323 | val acc 0.373 | val F1 0.315
epoch  2 | loss 1.193 | train acc 0.519 | val acc 0.525 | val F1 0.528
epoch  3 | loss 1.103 | train acc 0.561 | val acc 0.627 | val F1 0.582
epoch  4 | loss 1.001 | train acc 0.627 | val acc 0.653 | val F1 0.619
epoch  5 | loss 0.934 | train acc 0.669 | val acc 0.703 | val F1 0.697
epoch  6 | loss 0.840 | train acc 0.741 | val acc 0.771 | val F1 0.774
epoch  7 | loss 0.769 | train acc 0.778 | val acc 0.797 | val F1 0.784
epoch  8 | loss 0.712 | train acc 0.808 | val acc 0.780 | val F1 0.779
epoch  9 | loss 0.696 | train acc 0.846 | val acc 0.780 | val F1 0.762
epoch 10 | loss 0.650 | train acc 0.846 | val acc 0.763 | val F1 0.761
epoch 11 | loss 0.735 | train acc 0.789 | val acc 0.831 | val F1 0.822
epoch 12 | loss 0.641 | train acc 0.875 | val acc 0.788 | val F1 0.766
epoch 13 | loss 0.580 | train acc 0.894 | val acc 0.839 | val F1 0.827
epoch 14 | loss 0.587 | train acc 0.886 | val acc 0.864 | val F1 0.864
epoch 15 | loss 0.569 | train acc 0.890 | val acc 0.822 | val F1 0.820
epoch 16 | loss 0.551 | train acc 0.922 | val acc 0.907 | val F1 0.906
epoch 17 | loss 0.521 | train acc 0.949 | val acc 0.915 | val F1 0.914
epoch 18 | loss 0.494 | train acc 0.952 | val acc 0.847 | val F1 0.837
epoch 19 | loss 0.489 | train acc 0.952 | val acc 0.890 | val F1 0.888
epoch 20 | loss 0.481 | train acc 0.952 | val acc 0.915 | val F1 0.913
epoch 21 | loss 0.476 | train acc 0.962 | val acc 0.907 | val F1 0.905
epoch 22 | loss 0.487 | train acc 0.956 | val acc 0.898 | val F1 0.895
epoch 23 | loss 0.484 | train acc 0.964 | val acc 0.890 | val F1 0.886
epoch 24 | loss 0.467 | train acc 0.971 | val acc 0.907 | val F1 0.903
epoch 25 | loss 0.465 | train acc 0.973 | val acc 0.907 | val F1 0.904
```





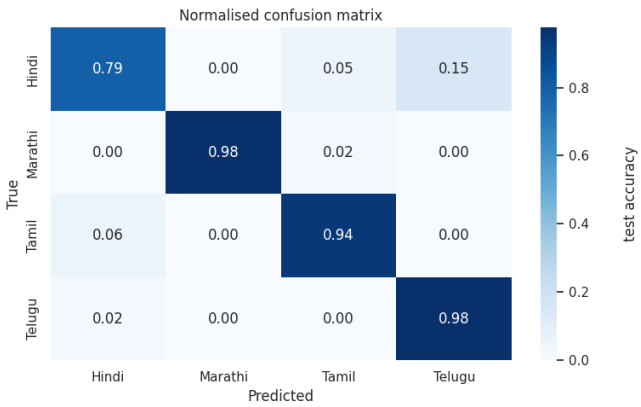
4. Evaluation of the CRNN classifier (test set)

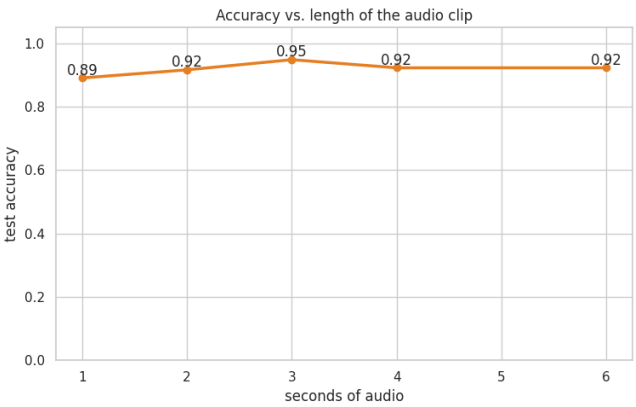
```
Test accuracy: 0.9231 | Macro-F1: 0.9215

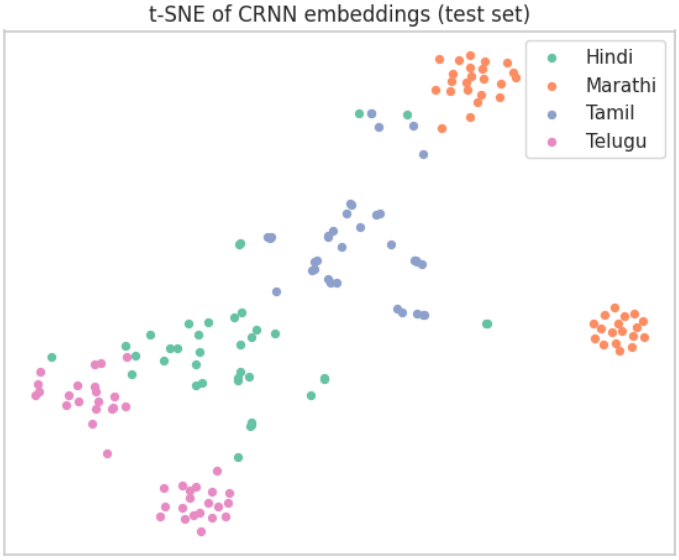
              precision    recall  f1-score   support

       Hindi     0.9118    0.7949    0.8493        39
    Marathi      1.0000    0.9756    0.9877        41
       Tamil     0.9167    0.9429    0.9296        35
      Telugu     0.8696    0.9756    0.9195        41

    accuracy                         0.9231       156
   macro avg     0.9245    0.9222    0.9215       156
weighted avg     0.9250    0.9231    0.9221       156
```

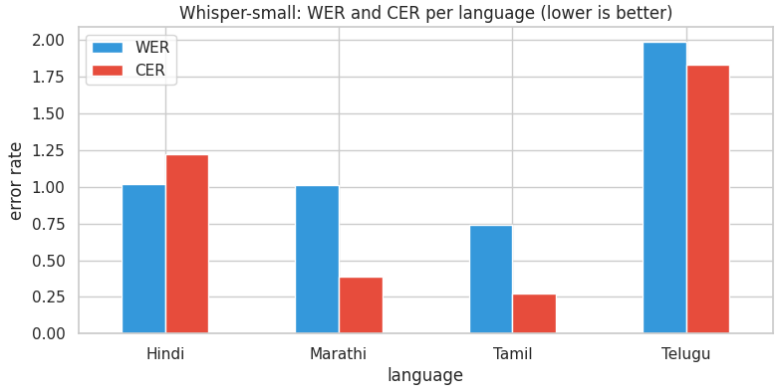






5. Speech-to-Text with Whisper-small

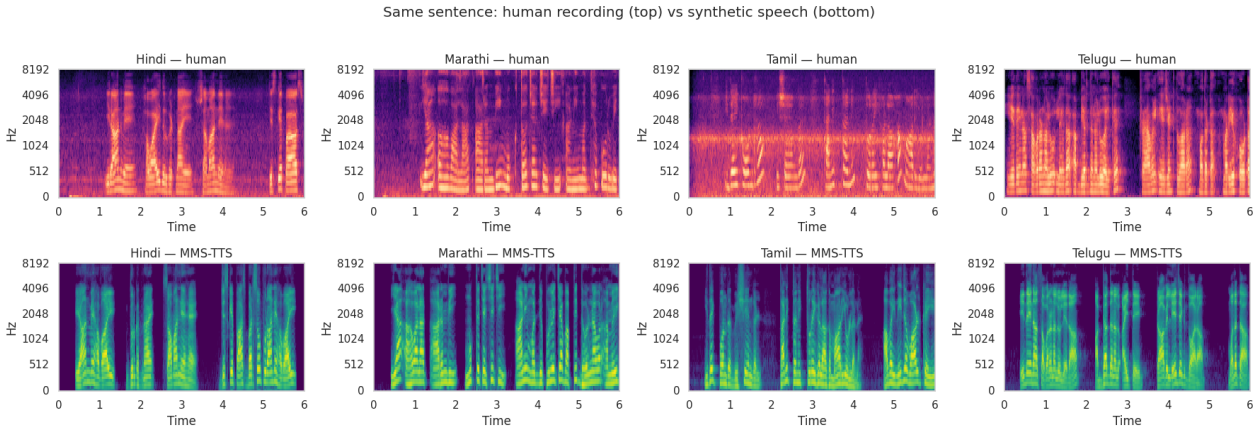
language	WER	CER
Hindi	1.019	1.220
Marathi	1.011	0.391
Tamil	0.741	0.275
Telugu	1.989	1.830
Average	1.190	0.929

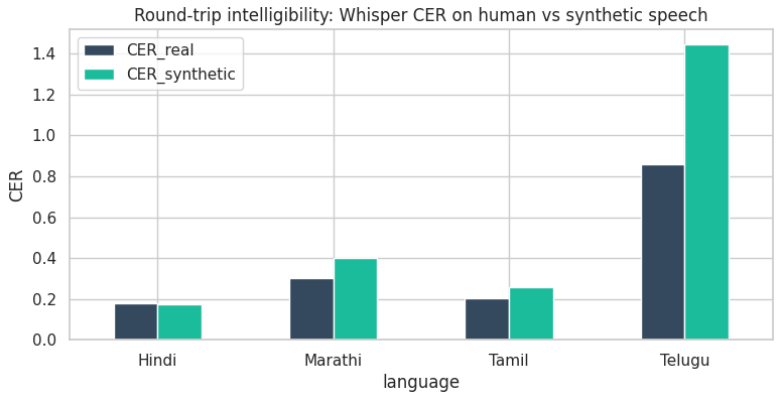


	language	reference	whisper	CER
0	Hindi	इरान में हवाई दुर्घटनाएं सामान्य ह...	2. వార్ల.....	1.37931
1	Hindi	पुलिस के मुताबिक फ़ोटोग्राफर को टक...	PULIS KE MUTABIK PHOTOGRAPHER KOTA...	0.920792
10	Marathi	या अहवालात आरंभी मुक्त चर्चेची आणि...	या अहवालात आरंभी मुक्तचर्चेजी अनी ...	0.300885
11	Marathi	टक्कर नक्की किती मोठी होती आणि त्य...	तक्कर नक्की की भी भी भी तो ती और त...	0.376147
20	Tamil	இதைப்போன்ற விஷயங்கள் தனித்தனித் து...	இதைப் போன்ற விஷயங்கள் தனித்தனு துற...	0.20155
21	Tamil	பல பொதுவான அமைப்புகள் உதாரணமாக ஏபி...	பில பொதுவான அமைப்புகள் தாரணமாக அப்...	0.188034
30	Telugu	ఏదైనా సవరణలు లేదా అభ్యర్థనలు అయితే...	यह आँ चवरनलू लेडा अप्यर्दनलू आप यह...	0.858696
31	Telugu	పోలాండ్ పురుషుల దృష్టి లోపం ఉన్న స...	Pouland Purshila Dhrushhti ninina...	0.894273

6. Text-to-Speech with MMS-TTS (VITS)

	language	real_sec	synthetic_sec	CER_real	CER_synthetic
0	Hindi	15.508	12.544	0.179	0.172
1	Marathi	13.856	9.184	0.301	0.398
2	Tamil	14.088	10.448	0.202	0.256
3	Telugu	7.424	9.136	0.859	1.446





7. Complete pipeline demo: identify → transcribe → speak back

```
True language: Tamil   Predicted: Tamil (89.4%)
Transcript:
```

இதைப் போன்ற வழியுங்கள் தனித்தனி துறைகளாக மாறிவில்லை என்று அவன் நிஜ வாழ்க்கை பிரச்சனைகளைக் காணத்தியுப்புகளை வழங்கு வதில் கவனம் சரித்துக்கொண்டு இருந்துவிடுகிறேன் என்று

```
Input audio and spoken-back (synthetic) audio are played with an audio player in the notebook.
```

8. Results summary

```
Audio classification (CRNN, 4 s clips)
  Test accuracy : 0.9231
  Macro-F1      : 0.9215

Speech-to-text (Whisper-small, language from CRNN)
  Average WER   : 1.190
  Average CER   : 0.929

Text-to-speech (MMS-TTS)
  Round-trip CER: human 0.385 vs synthetic 0.568
```

Conclusion:

- The CRNN identified the spoken language with a test accuracy of 92.31% and a macro-F1 of 0.9215. Marathi was recognised best (F1 0.988); Hindi had the lowest recall (0.795) and Telugu the lowest precision (0.870).
- Whisper-small gave an average WER of 1.190 and CER of 0.929 on the 40 test clips. A WER above 1 is possible because insertions (for example repeated tokens) are counted as errors. Tamil (CER 0.275) and Marathi (CER 0.391) were best, while Hindi and especially Telugu produced repetitive or wrong-script output. CER is the more meaningful metric here since Indian-language words are long.

- MMS-TTS speech was most intelligible for Hindi (round-trip CER 0.172 vs 0.179 for the human recording). Marathi and Tamil were somewhat worse, and Telugu was poor (1.446), which is largely limited by Whisper-small's weak Telugu recognition (human CER 0.859). The average round-trip CER was 0.568 for synthetic speech against 0.385 for human speech.
- Together the three models form a working loop: identify the language, transcribe the speech and speak it back.